
Scenario 1: Book Haven – Online Bookstore

Book Haven is a popular bookstore. They have decided to launch an online shopping portal to automate book orders. Customers can buy books and avail discounts based on the payment method they choose (Credit Card, Debit Card, NetBanking).

Your task is to write a Java program that stores book order details, validates book IDs, calculates discounts, and generates the final bill.

Component Specification: Book (Model Class)

- **Attributes**
 - String bookId
 - String bookName
 - double pricePerUnit
 - int quantity
 - **Methods**
 - Getters and setters for all attributes
 - Four-argument constructor: bookId, bookName, pricePerUnit, quantity
-

Component Specification: BookOrder

- **Attributes**
 - List<Book> bookList
 - double discount
 - **Methods**
 - public void bookBooking(Book bookObject[]) → Adds array of Book objects to bookList
 - public void findDiscount(String paymentMethod) → Sets discount based on payment method
 - public double calculateTotalBill() → Calculates final bill
-

Requirement 1: Validate bookId

- **Utility Class**
 - public boolean validateBookId(String bookId)
 - BookId must start with "BOOK" followed by 3 digits (e.g., BOOK123).
 - If invalid, throw InvalidBookIdException with message "Book Id <<bookId>> is not valid".
-

Discount Table

Payment Method	Discount (%)
CreditCard	20
DebitCard	15
NetBanking	10

Requirement 4: Bill Calculation

- Total Price = sum of (pricePerUnit * quantity) for all books
 - Bill Amount = Total Price - (Total Price * discount / 100)
-

Sample Input/Output 1

Enter the total number of books needed

2

Enter the Book details

BOOK101:HarryPotter:500:2

BOOK102:PercyJackson:400:1

Enter the payment method

CreditCard

Total Bill Amount: 1120.00

Sample Input/Output 2

Enter the total number of books needed

1

Enter the Book details

XYZ999:LordOfTheRings:600:1

Book Id XYZ999 is not valid

Your booking is empty

Scenario 2: Gadget World – Online Electronics Store

Gadget World sells electronic gadgets. They want to automate gadget orders with discounts based on the payment method (AmazonPay, Paytm, GooglePay).

Your task is to write a Java program that stores gadget order details, validates gadget IDs, calculates discounts, and generates the final bill.

Component Specification: Gadget (Model Class)

- **Attributes**
 - String gadgetId
 - String gadgetName
 - double pricePerUnit
 - int quantity
 - **Methods**
 - Getters and setters for all attributes
 - Four-argument constructor: gadgetId, gadgetName, pricePerUnit, quantity
-

Component Specification: GadgetOrder

- **Attributes**
 - List<Gadget> gadgetList

- double discount
 - **Methods**
 - public void gadgetBooking(Gadget gadgetObject[]) → Adds array of Gadget objects to gadgetList
 - public void findDiscount(String paymentMethod) → Sets discount based on payment method
 - public double calculateTotalBill() → Calculates final bill
-

Requirement 1: Validate gadgetId

- **Utility Class**
 - public boolean validateGadgetId(String gadgetId)
 - GadgetId must start with "GAD" followed by 4 digits (e.g., GAD1234).
 - If invalid, throw InvalidGadgetIdException with message "Gadget Id <<gadgetId>> is not valid".
-

Discount Table

Payment Method	Discount (%)
AmazonPay	25
GooglePay	20
Paytm	15

Requirement 4: Bill Calculation

- Total Price = sum of (pricePerUnit * quantity) for all gadgets
 - Bill Amount = Total Price - (Total Price * discount / 100)
-

Sample Input/Output 1

Enter the total number of gadgets needed

2

Enter the Gadget details

GAD1001:Smartphone:15000:1

GAD1002:Headphones:2000:2

Enter the payment method

AmazonPay

Total Bill Amount: 15250.00

Sample Input/Output 2

Enter the total number of gadgets needed

1

Enter the Gadget details

ABC123:Tablet:12000:1

Gadget Id ABC123 is not valid

Your booking is empty

Scenario 3: Ocean Breeze Fish Market

Ocean Breeze Fish Market is a popular seafood shop. The management wants software to store fish details, calculate bills, and apply discounts based on coupon codes.

Component Specification: Customer (Model Class)

- **Attributes**

- String customerName
- long mobileNumber
- double discountPercentage

- **Methods**

- Getters and setters for all attributes
- Three-argument constructor: customerName, mobileNumber, discountPercentage

Component Specification: CustomerOrder

- **Attributes**

- Map<String, Double> fishMap

- **Methods**

- public void addFish(String fishName, double quantity) → Adds fish name and quantity to fishMap
 - public double calculateBill() → Calculates total bill (without discount)
-

Requirement 1: Validate Coupon Code

- **CustomerUtility Class**

- public boolean validateCouponCode(String couponCode)
 - Coupon code must start with "FM" followed by 3 digits (digits can take values 1–7).
 - If invalid, throw InvalidCouponCodeException with message:
"This coupon code <<couponCode>> is not eligible for discount"
-

Requirement 2: Find Discount Percentage

- public double findDiscountPercentage(String couponCode)
- Discount percentage = sum of digits in coupon code.

Example: FM523 → 5 + 2 + 3 = 10% discount.

Fish Price Table

Fish Name	Price (per kg)
Salmon	600
Tuna	450
Mackerel	300
Sardine	200

Fish Name	Price (per kg)
Prawns	700
Crab	800
Lobster	1200

Sample Input/Output 1

Enter the customer name

John

Enter the customer mobile number

987654321

Enter the number of fishes

2

Enter the fish details

Salmon:1.5

Prawns:2

Enter the Coupon Code

FM523

Please pay Rs 2235.00 after 10.00 percentage discount

Sample Input/Output 2

Enter the customer name

Alice

Enter the customer mobile number

912345678

Enter the number of fishes

1

Enter the fish details

Crab:1

Enter the Coupon Code

FM888

This coupon code FM888 is not eligible for discount

Please pay Rs 800.00 after 0.00 percentage discount

Scenario 4: Green Valley Vegetable Mart

Green Valley Vegetable Mart sells fresh vegetables. They want software to automate billing and discounts using coupon codes.

Component Specification: Customer (Model Class)

- **Attributes**
 - String customerName
 - long mobileNumber
 - double discountPercentage
 - **Methods**
 - Getters and setters for all attributes
 - Three-argument constructor: customerName, mobileNumber, discountPercentage
-

Component Specification: CustomerOrder

- **Attributes**
 - Map<String, Double> vegetableMap
 - **Methods**
 - public void addVegetable(String vegetableName, double quantity) → Adds vegetable name and quantity to vegetableMap
 - public double calculateBill() → Calculates total bill (without discount)
-

Requirement 1: Validate Coupon Code

- **CustomerUtility Class**
 - public boolean validateCouponCode(String couponCode)
 - Coupon code must start with "VG" followed by 3 digits (digits can take values 1–9).
 - If invalid, throw InvalidCouponCodeException with message:
"This coupon code <<couponCode>> is not eligible for discount"
-

Requirement 2: Find Discount Percentage

- public double findDiscountPercentage(String couponCode)
- Discount percentage = sum of digits in coupon code.

Example: VG234 → 2 + 3 + 4 = 9% discount.

Vegetable Price Table

Vegetable	Price (per kg)
Tomato	50
Potato	40
Onion	60
Carrot	80
Beans	100
Cauliflower	70
Spinach	30
Broccoli	120

Sample Input/Output 1

Enter the customer name

Meera

Enter the customer mobile number

998877665

Enter the number of vegetables

3

Enter the vegetable details

Tomato:2

Onion:1.5

Broccoli:1

Enter the Coupon Code

VG234

Please pay Rs 352.70 after 9.00 percentage discount

Sample Input/Output 2

Enter the customer name

Ravi

Enter the customer mobile number

987123456

Enter the number of vegetables

2

Enter the vegetable details

Potato:3

Spinach:2

Enter the Coupon Code

VG999

This coupon code VG999 is not eligible for discount

Please pay Rs 210.00 after 0.00 percentage discount
